

মডিউল ১০ঃ Disjoint Set Union >

মডিউল ১১ঃ DSU Cycle Detection এবং MST >

মডিউল ১৩ঃ বেসিক গ্রাফ রিক্যাপ >

মডিউল ১৪ঃ প্রব্লেম সল্ভিং গ্রাফ অ্যালগরিদম দিয়ে >

মডিউল ১৫ঃ প্রব্লেম সল্ভিং ২ গ্রাফ অ্যালগরিদম দিয়ে >

মডিউল ১৬ঃ বেসিক ডাইনামিক প্রোগ্রামিং >

মডিউল ১৭-০১ঃ ইনট্রডাকশন

মডিউল ১৭-০২ঃ ডাইনামিক প্রোগ্রামিং কি?

মডিউল ১৭-০৩ঃ ফ্যাক্টোরিয়াল নাম্বার

মডিউল ১৭-০৪ঃ Fibonacci Series

মডিউল ১৭-০৫ঃ Fibonacci Subproblem

মডিউল ১৭-০৬ঃ Fibonacci Memoization

মডিউল ১৭-০৭ঃ Bottom up approach in Fibonacci series

মডিউল ১৮ঃ Knapsack >

মডিউল ১৮-০১ঃ ইনট্রডাকশন

মডিউল ১৮-০২ঃ Knapsack

মডিউল ১৮-০২ঃ Knapsack

আমরা Knapsack বুঝার জন্য নিচে দেয়া প্রব্লেমটি সমাধান করবো।

Problem Link: <https://codeforces.com/group/MWSDmqGsZm/contest/223339/problem/U>

এই প্রব্লেম বলা হয়েছে, আপনাকে N items দেয়া হবে, প্রতিটি item এর একটি weight এবং value দেয়া থাকবে। একটি item কে আপনি শুধুমাত্র একবারই নিতে পারবেন। item গুলোকে রাখার জন্য আপনার কাছে একটি ব্যাগ আছে এবং সেই ব্যাগটিতে একটি নির্দিষ্ট পরিমাণ পর্যন্ত weight বহন করতে পারে আর সেই weight টি হল W।

weight = [2,3,4,5]

value = [1,3,5,3]

W = 10

Item Number	1	2	3	4
weight	2	3	4	5
value	1	3	5	3

```

1 int knapsack(int n, int weight[], int value[], int W){
2
3     if (n == 0 || W == 0)
4
5         return 0;
6
7     if (weight[n - 1] <= W)
8
9     {
10
11         // duita option
12
13         // niye dekhbo, na niye dekhbo
14
15         int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
16
17         int op2 = knapsack(n - 1, weight, value, W);
18
19         return max(op1, op2);
20
21     }
22
23     else
24
25     {
26
27         // ekta option
28
29         // na niyei dekhte hobe
30
31         int op2 = knapsack(n - 1, weight, value, W);
32
33         return op2;
34
35     }
36
37 }
```

Knapsack এ আমাদের কাছে দুইটি অপশন থাকে একটি হলো নিয়ে দেখবো এবং আরেকটি হলো না নিয়ে দেখবো।

```

1 if (weight[n - 1] <= W)
```

যখন item টি weight আমাদের ব্যাগের weight W এর সমান অথবা ছোট হবে তখনই আমরা item টিকে নিয়ে অথবা না নিয়ে দেখতে পারবো।

```

1 // op1 e niye dekhbo item ti ke,
2
3 int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
4
5
6
7 // op2 te na niye dekhbo item ti ke,
8
9 int op2 = knapsack(n - 1, weight, value, W);
```

op1 তে আমরা item টিকে নিয়েদেখবো তাই knapsack function টিকে recursive কল করতেছি সাথে এই item টির value যোগ করতেছি এবং কল করা সময়

W- weight[n-1] এর মাধ্যমে নিয়া item টির weight বাদ দিয়েছি ব্যাগের weight থেকে কারন আমরা knapsack function টির base case দিয়েছে এমনভাবে যে, item শেষ হয়ে গেলে অথবা ব্যাগের মধ্যে অবশিষ্ট জায়গা না থাকলে return 0 করা হবে।

op2 তে আমরা item টিকে নিচ্ছি না তাই knapsack function কে recursive কল করার সময় শুধুমাত্র item number কে change করে কল করতেনি।

```
1 return max(op1, op2);
```

এরপর op1 এবং op2 থেকে যেই অপশনটি maximum value দিবে আমরা সেইটিকে নিচ্ছি return করতেনি।

```
1 else
2
3 {
4
5 // ekta option
6
7 // na niyei dekhite hobe
8
9 int op2 = knapsack(n - 1, weight, value, W);
10
11 return op2;
12
13 }
```

যেহেতু item টি weight আমাদের ব্যাগের weight W এর সমান অথবা ছোট না তাই item টিকে নিতে পারবো না। এইজন্য item টিকে নিচ্ছি না তাই knapsack function কে recursive কল করার সময় শুধুমাত্র item number কে change করে কল করতেনি। আর সেই অপশনটিকে return করতেনি।

fn(4,10) starts running

4,10

Item Number	1	2	3	4
weight	2	3	4	5
value	1	3	5	3

যেহেতু W=10 (ব্যাগের weight) তাই আমরা if condition এ মধ্যো যেতে পারবো এবং, নিয়ে অথবা না নিয়ে দেখতে পারবো।

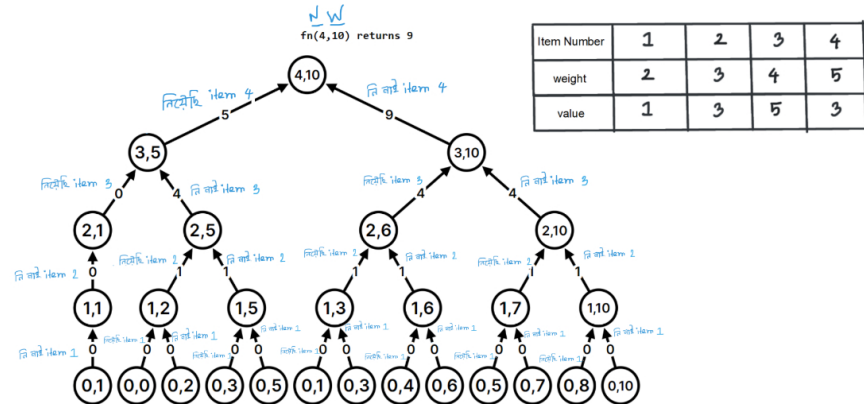
fn(3,5) starts running

4,10

3,5

Item Number	1	2	3	4
weight	2	3	4	5
value	1	3	5	3

নিয়ার ফলে knapsack function টি recursive কল হয়েছে এবং W- item 4 এর weight বাদ দেয়া হয়েছে।



এইভাবে বাকি কল গুলোকে complete করা পর এইভাবে maximum value 9 বের ফাইন্ড করে ফেলবো Knapsack দিয়ে।

<

Previous
মডিউল ১৮-০৪ ইনস্ট্রাকশন

Next
মডিউল ১৮-২৪ Knapsack Recursive Approach

>

Last updated September 9, 2024

